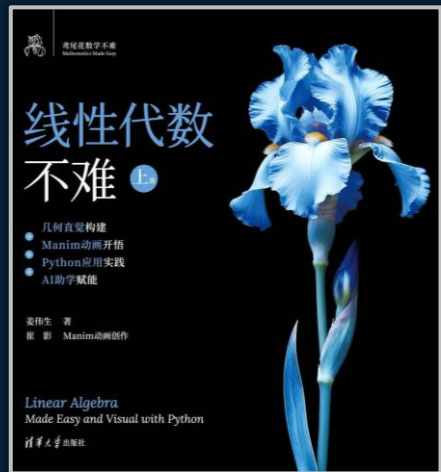


10 回归

10.1 从投影到回归

从一条直线到正交投影——用格拉姆矩阵与广义逆，推导一元线性回归的最优参数



鸢尾花书·《线性代数不难》 | 开源代码与书稿: <https://github.com/Visualize-ML>

本节你将掌握的核心技能

1 一条直线拟合数据，表示变量之间的线性关系

2 截距决定上下位置，斜率决定倾斜程度

3 预测值向量是目标向量在解释变量张成空间的正交投影

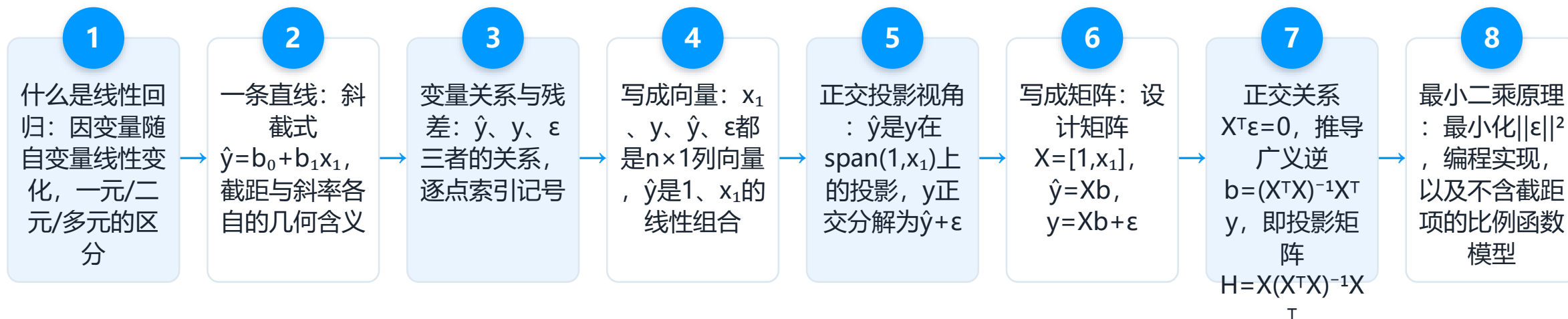
4 利用格拉姆矩阵求解最优参数

5 广义逆在超定方程组中的作用，并能用于回归系数求解

6 预测值与残差都由投影矩阵表达，且彼此正交

7 最小二乘原理：最小化残差平方和

从一条直线，到正交投影，再到最小二乘



本章每个回归话题，建议从这几方面看

- ① 几何视角：预测值是真实值向量在解释变量张成空间中的正交投影
- ② 代数视角：设计矩阵 $\mathbf{X}$ 、格拉姆矩阵 $\mathbf{X}^T \mathbf{X}$ 、广义逆 $(\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T$
- ③ 最优化视角：最小二乘——最小化残差平方和 $\|\boldsymbol{\epsilon}\|^2$
- ④ 统计视角：残差、拟合优度、显著性检验，留给《概率统计不难》深入讨论
- ⑤ 推广路径：从一元到二元、多元线性回归，再到非线性的多项式回归
- ⑥ 编程视角：手写矩阵公式与调用Scikit-Learn，结果完全一致
- ⑦ 应用场景：预测、趋势拟合、变量关系解释，是数据分析最基础的工具之一

什么是线性回归？

回归，是描述因变量如何随自变量变化的数学模型

回归：建立数学模型，描述因变量(dependent variable)如何随一个或多个自变量(independent variable)变化

线性回归核心假设：自变量对因变量的影响是线性累加的，而非指数、乘积等更复杂的非线性关系

一元线性回归(univariate)：只有一个自变量参与建模，例如用身高预测体重

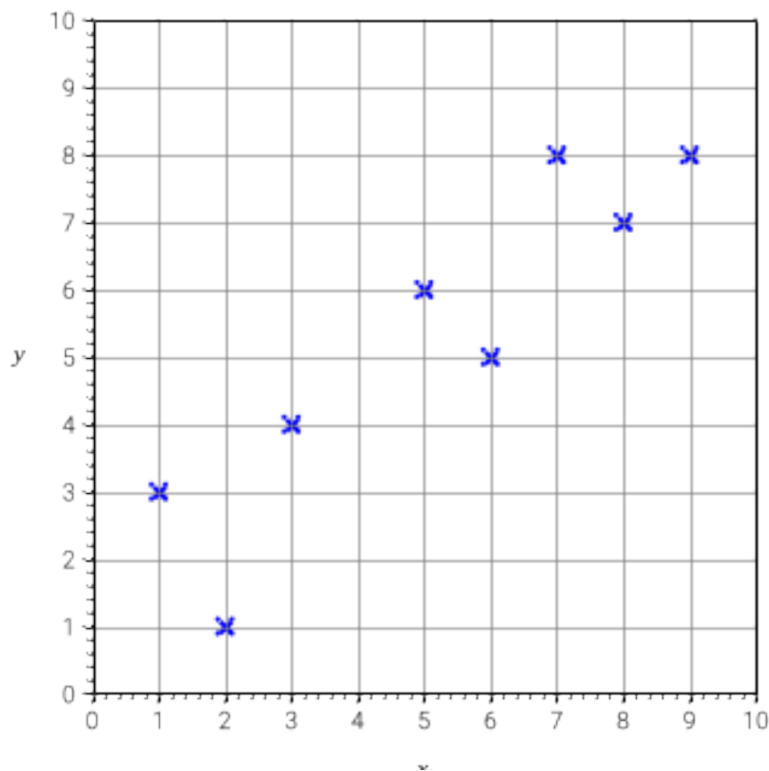
二元、多元回归：两个或更多自变量参与建模，“元”表示自变量个数

下一节将介绍多项式回归——一种常见的非线性回归模型

注意：线性回归揭示的只是相关性，不代表因果关系，背后可能存在第三方因素或纯属巧合

图1 • 呈现线性关系的散点数据

- 9个散点大致呈线性趋势，想找一条直线尽量贴近每个点
- x_1 常称自变量、解释变量、回归元； y 常称因变量、被解释变量、回归子



红色直线，写成斜截式

$$\hat{y} = b_0 + b_1 x_1$$

- b_0 为截距(intercept), b_1 为斜率(slope)——这就是平面直线的斜截式
- 把 b_0 写在 b_1 前面, 是为了配合后文参数列向量 $\mathbf{b}=[b_0, b_1]^T$ 的次序
- $\hat{y}$ 读作y-hat: y 是散点的真实纵轴值, $\hat{y}$ 是直线给出的估计值、预测值

图2 • 一条红色直线拟合散点

- 红色直线 $\hat{y}=b_1x_1+b_0$ 尽量贴近蓝色散点，即为一元线性回归的拟合直线

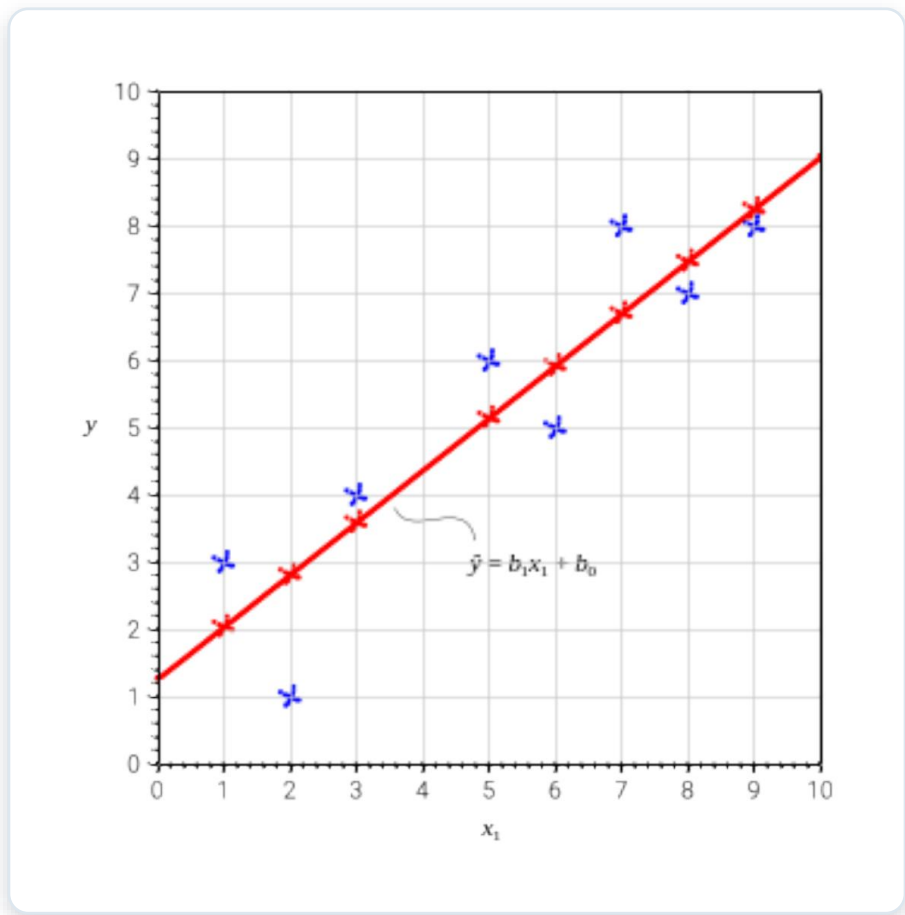
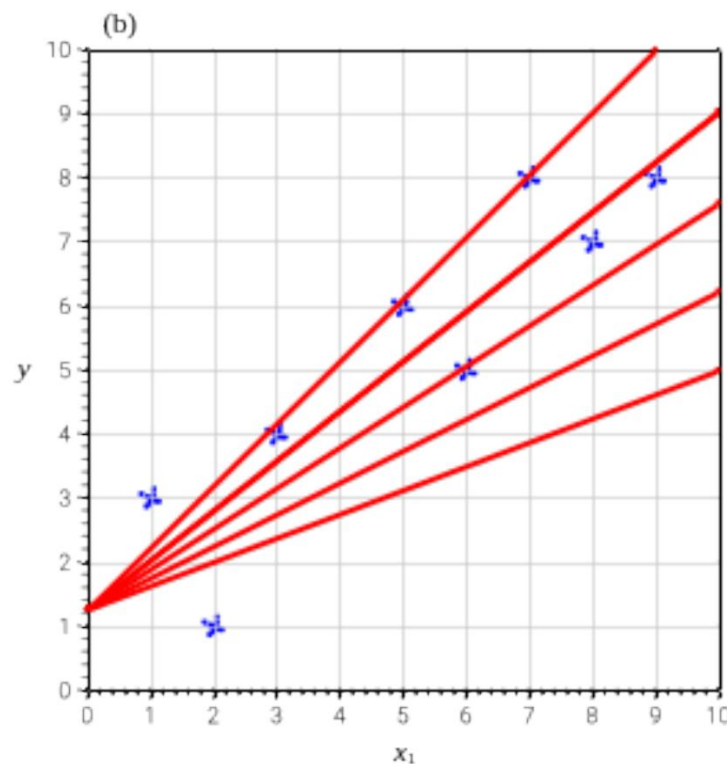
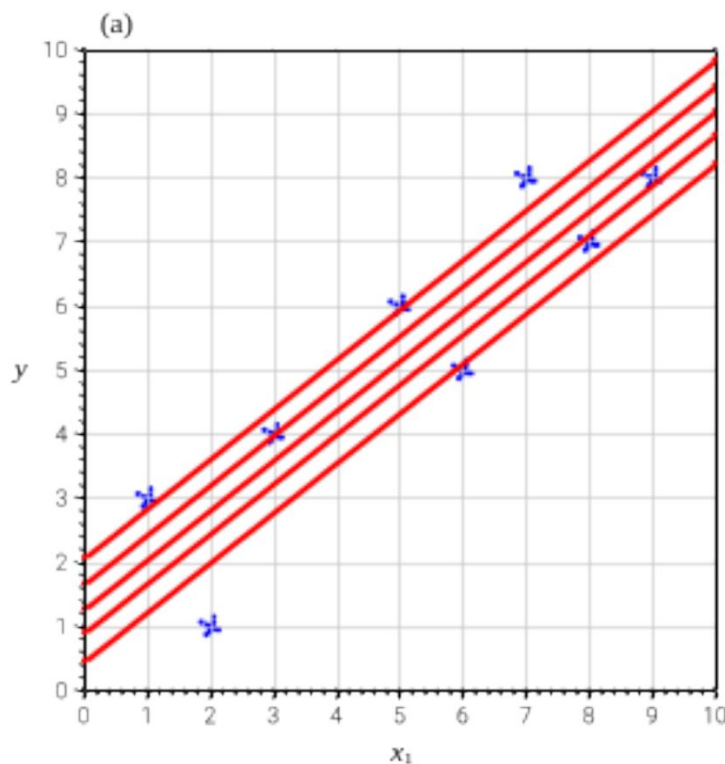


图3 • (a)截距决定上下位置, (b)斜率决定倾斜程度

- (a) 斜率不变, 改变截距 b_0 使直线整体上下平移, 不改变倾斜程度
- (b) 截距不变, 改变斜率 b_1 使直线以截距为支点旋转, 绝对值越大越陡峭



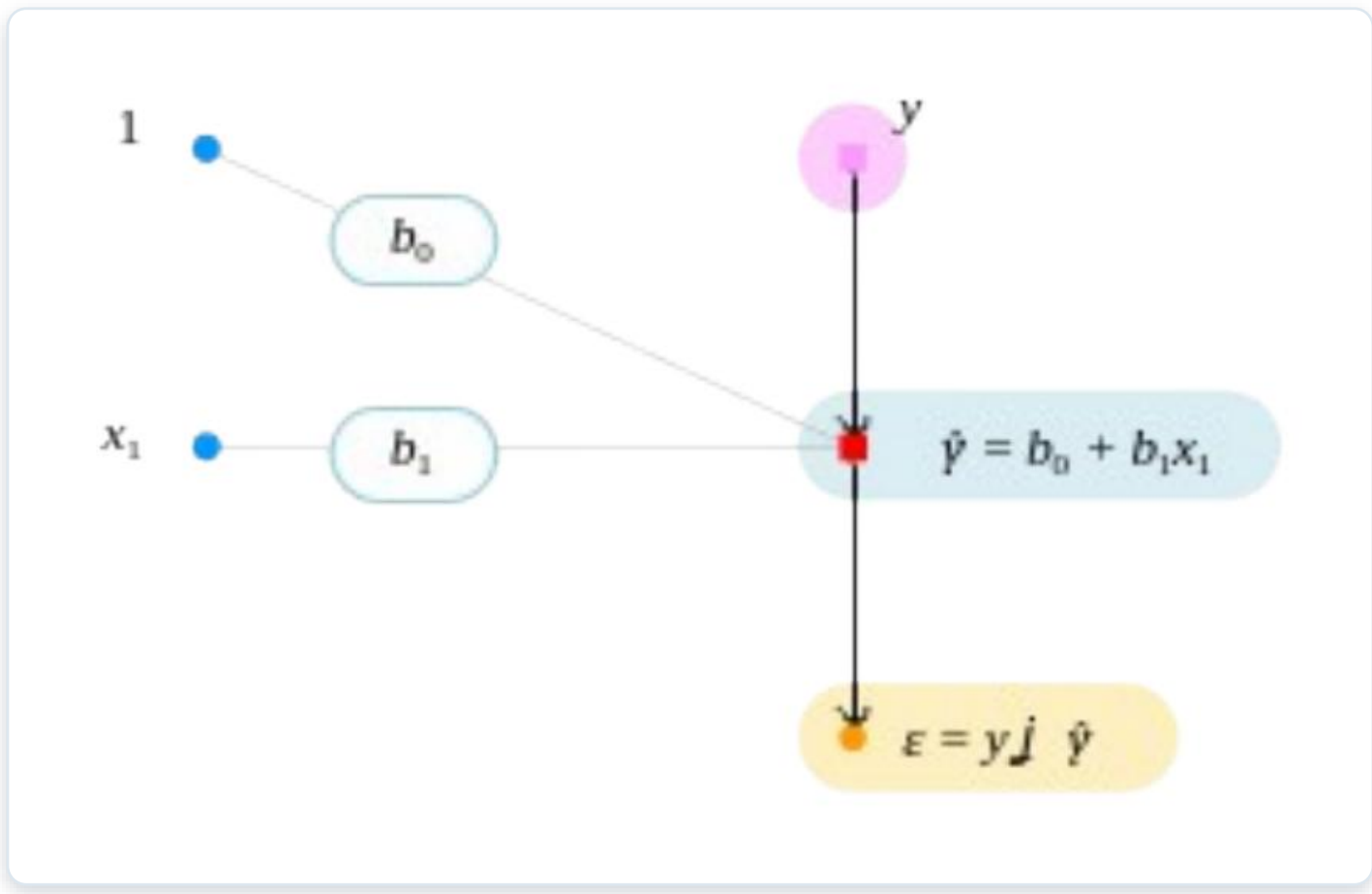
残差项，是真实值与预测值之差

$$\varepsilon = y - \hat{y} = y - (b_0 + b_1 x_1)$$

- ε 也叫残差(residuals)、误差项(error term)、干扰项、噪音项
- 真实的 y 、预测的 $\hat{y}$ 之间的差距，就是我们希望尽量缩小的目标

图4 • 一元线性回归变量关系

- 1、 x_1 分别乘以参数 b_0 、 b_1 后相加，得到预测值 $\hat{y}$ ； y 与 $\hat{y}$ 之差得到误差 ε



第*i*个数据点的预测值、误差、真实值

$$y^{(i)} = b_0 + b_1 x_1^{(i)} + \varepsilon^{(i)}$$

- 预测值: $\hat{y}^{(i)} = b_0 + b_1 x_1^{(i)}$; 误差: $\varepsilon^{(i)} = y^{(i)} - \hat{y}^{(i)} = b_0 + b_1 x_1^{(i)} - \hat{y}^{(i)}$
- 真实值 $y^{(i)}$ 和估计值 $\hat{y}^{(i)}$ 一般并不贴合, 两者纵轴高度差就是残差

图5 • 一元线性回归中的真实值、估计值、误差

- 黄色高亮点索引为 i ，横轴 $x_1^{(i)}$ 、纵轴真实值 $y^{(i)}$ ；红线上对应估计值 $\hat{y}^{(i)}$

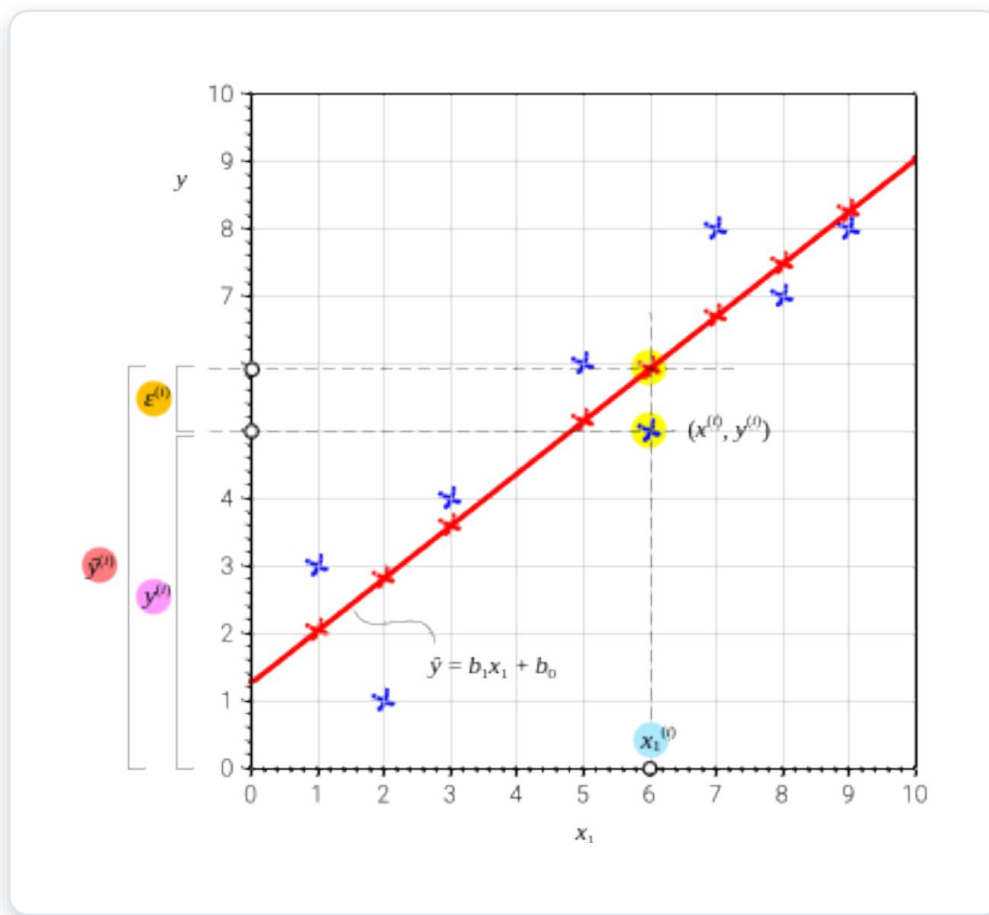
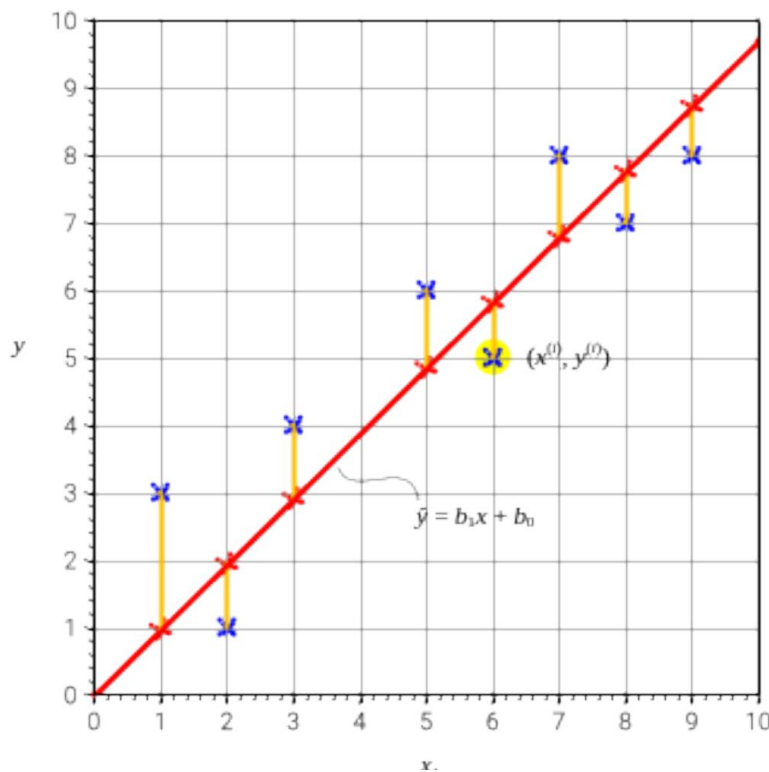


图6 • 一元线性回归中所有散点的误差（橙色线段）

- 橙色竖直线段就是每个散点的残差；让所有残差尽量小，红线才最贴合散点



把散点坐标、估计值、误差都写成 $n \times 1$ 列向量

$x_1, y, \hat{y}, \varepsilon$ 均为 $n \times 1$ 列向量

- x_1 对应图6中横轴坐标； y 对应蓝色 $\times$ 纵轴坐标； $\hat{y}$ 对应红色 $\times$ 纵轴坐标
- ε 对应图6中橙色线段——第 i 个分量为 $x_1^{(i)}, y^{(i)}, \hat{y}^{(i)}, \varepsilon^{(i)}$

图1中自变量、因变量数据列向量的具体数值

$$\mathbf{x}_1 = [1, 2, 3, 5, 6, 7, 8, 9]^T, \quad \mathbf{y} = [3, 1, 4, 6, 5, 8, 7, 8]^T$$

- 两个长度为9的列向量，逐个分量一一对应图1中的9个散点

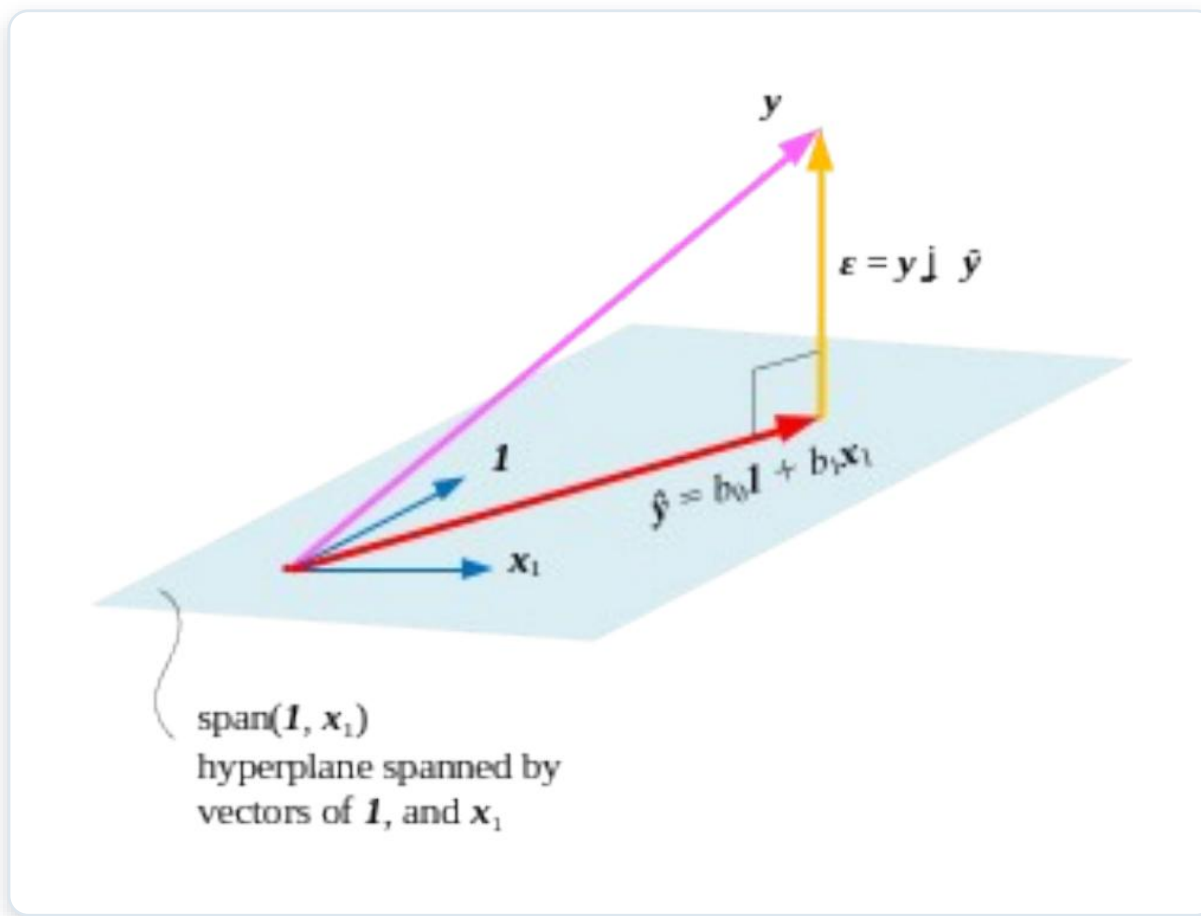
$\hat{y}$ 是1、 x_1 的线性组合

$$\hat{y} = b_0 \mathbf{1} + b_1 x_1$$

- $\mathbf{1}$ 为和 x_1 形状相同的全1列向量——这就是本书反复出现的线性组合
- 也就是说, $\hat{y}$ 在1、 x_1 张成的平面 $\text{span}(\mathbf{1}, x_1)$ 中! 这里默认1、 x_1 线性无关

图7 • 向量 y 向平面 $\text{span}(1, x_1)$ 投影，一元线性回归

- 真实值 y 在 $\text{span}(1, x_1)$ 之外； y 正交投影到平面上得到 $\hat{y}$ ，垂直分量为 ε



真实值 y 正交分解为预测值 $\hat{y}$ 与误差 ε

$$y = \hat{y} + \varepsilon = b_0 \mathbf{1} + b_1 \mathbf{x}_1 + \varepsilon$$

- 整理得误差向量： $\varepsilon = y - \hat{y} = y - (b_0 \cdot \mathbf{1} + b_1 \mathbf{x}_1)$
- 误差向量 ε 垂直于平面 $\text{span}(\mathbf{1}, \mathbf{x}_1)$ ——这一点后文求解最优参数时马上要用到

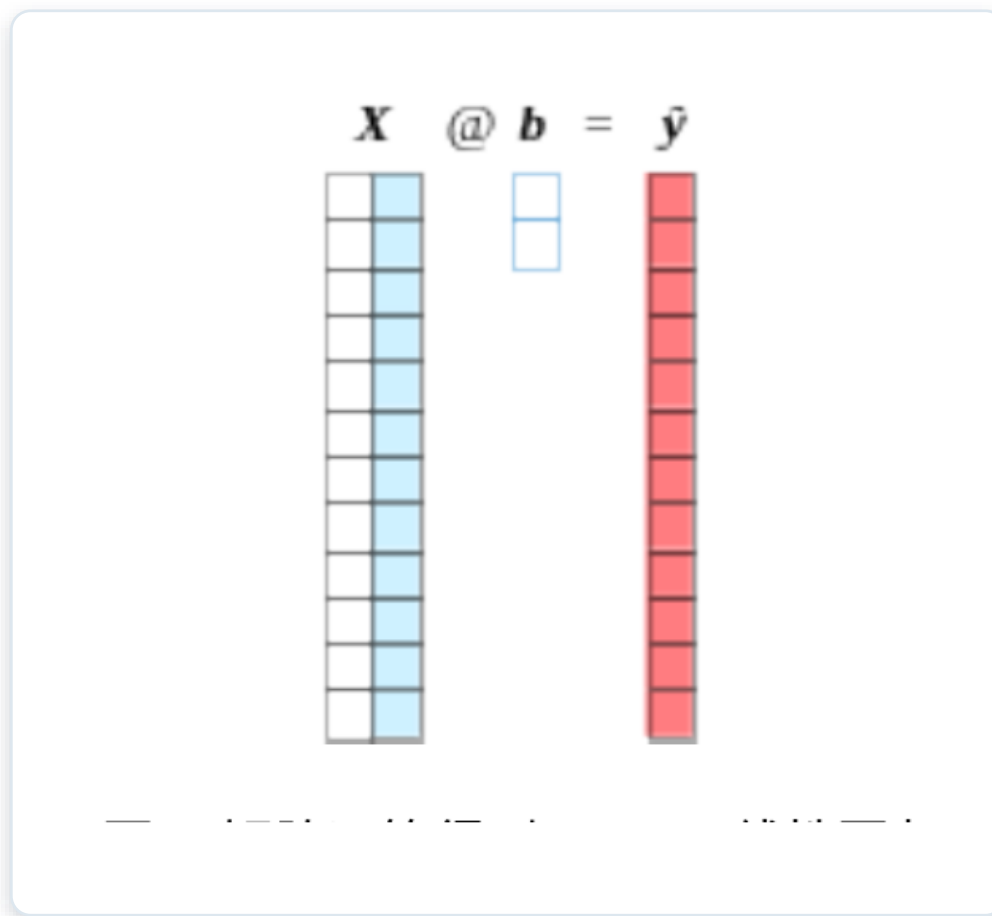
把线性组合写成矩阵乘法： $\hat{y} = Xb$

$$\hat{y} = [1 \ x_1] [b_0; b_1] = Xb$$

- $X=[1, x_1]$ ，把两个列向量拼成一个 $n \times 2$ 矩阵； $b=[b_0, b_1]^T$ 为参数列向量

图8 • 矩阵运算得到 $\hat{y}$ ，一元线性回归

- X 的每一行对应一个样本，与列向量 b 做矩阵乘法，逐行得到预测值 $\hat{y}$



X叫做设计矩阵 (design matrix)

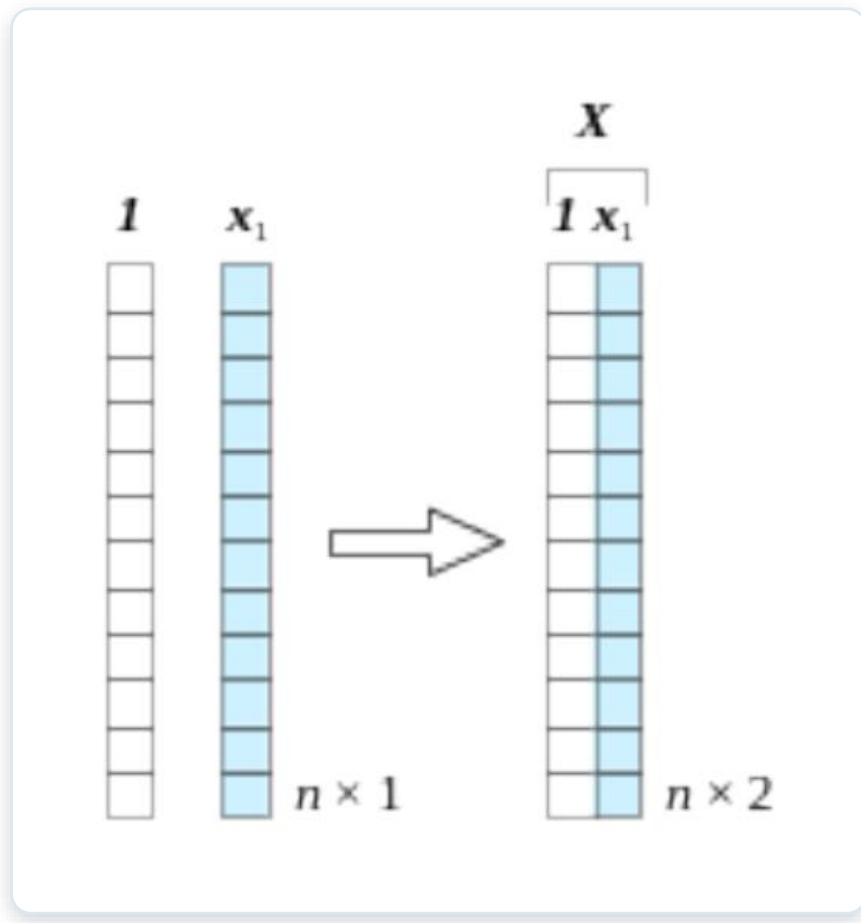
设计矩阵通常第一列添加全1列向量1，表示截距项；其余列为各自变量取值

一元线性回归、截距不为0时，X左侧需要添加全1列向量1

图1对应的设计矩阵X是一个8行2列矩阵，第一列全为1，第二列为 x_1 的9个取值

图9 • 设计矩阵 X 的构造方法，一元线性回归

- 把全1列向量 $\mathbf{1}$ ($n \times 1$) 与自变量列向量 x_1 ($n \times 1$) 左右拼接，得到 X ($n \times 2$)



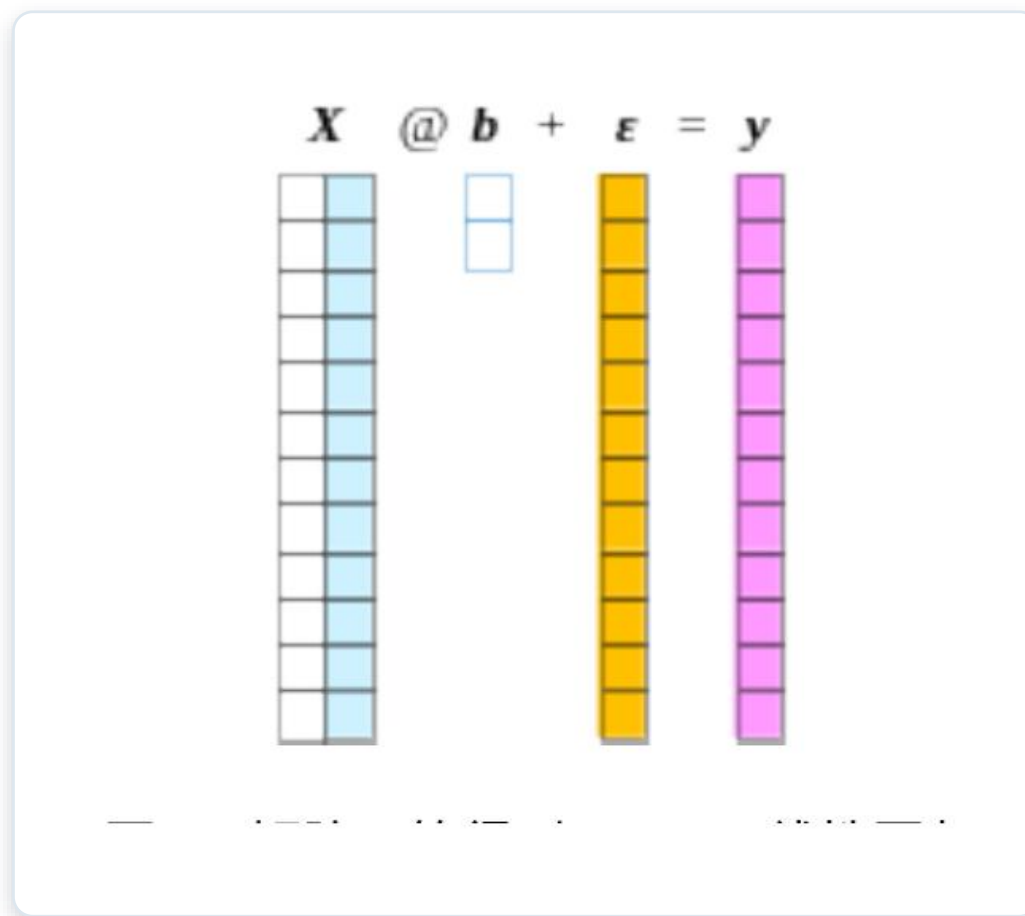
真实值 y 写成预测值 $\hat{y}$ 与误差 ε 之和

$$y = \hat{y} + \varepsilon = Xb + \varepsilon$$

- y 对应图6中蓝色 $\times$ 的纵轴坐标； $\hat{y}$ 对应红色 $\times$ 的纵轴坐标； ε 对应橙色线段

图10 • 矩阵运算得到y，一元线性回归

- Xb 加上误差列向量 ϵ ，逐行还原出真实值 y



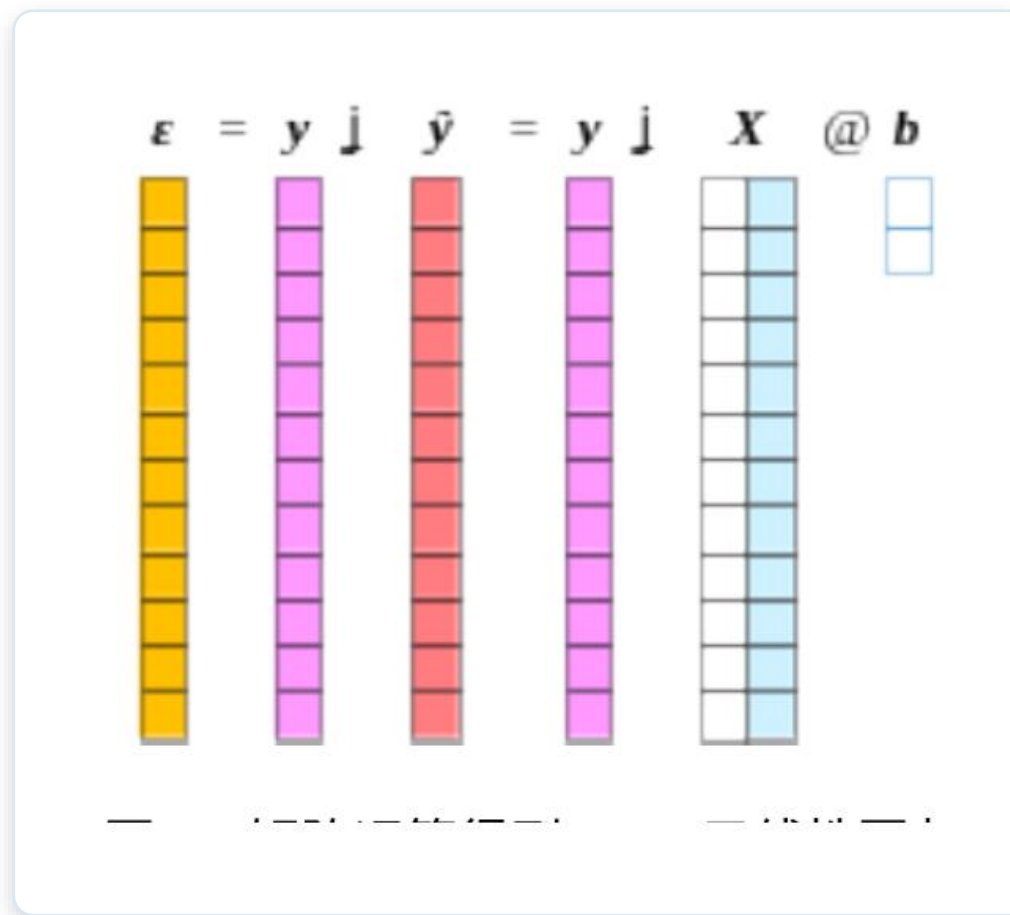
误差向量 ϵ ，垂直于 X 的每一列

$$\epsilon = y - \hat{y} = y - Xb$$

- 把前文投影结论用矩阵语言重新表达： ϵ 与设计矩阵 X 的每一列都正交

图11 • 矩阵运算得到 ϵ ，一元线性回归

- y 减去 Xb ，逐行得到残差列向量 ϵ



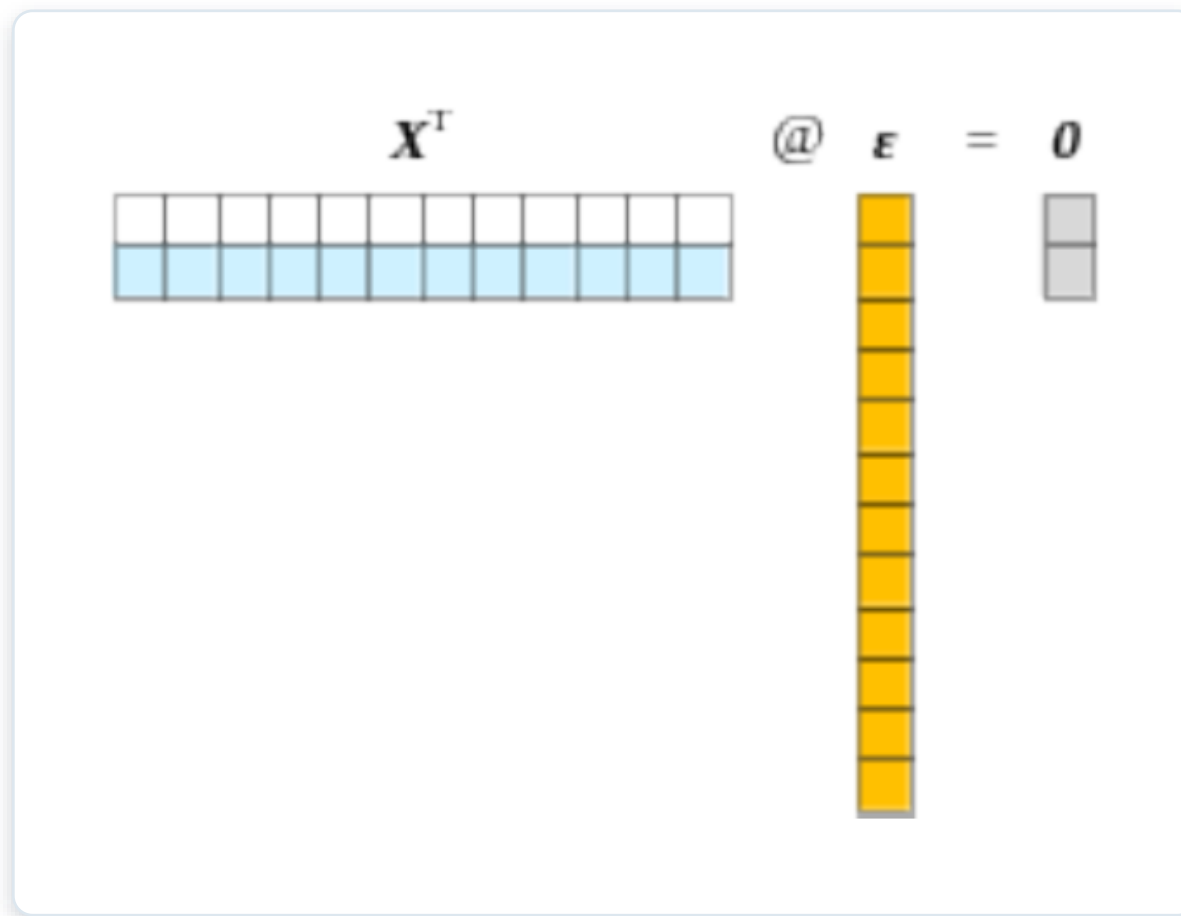
ϵ 垂直X每一列，等价于X转置乘以 ϵ 为0

$$\mathbf{1}^T \epsilon = 0, \quad \mathbf{x}_1^T \epsilon = 0 \implies \mathbf{X}^T \epsilon = 0$$

- 把两个正交条件写成一个矩阵乘法： $\mathbf{X}^T \epsilon = 0$ ——这一点上一章9.4、9.5已经讲过

图12 • ϵ 垂直X的每一列，一元线性回归

- X^T 与 ϵ 做矩阵乘法，结果为零向量——这正是 ϵ 垂直X每一列的矩阵表达



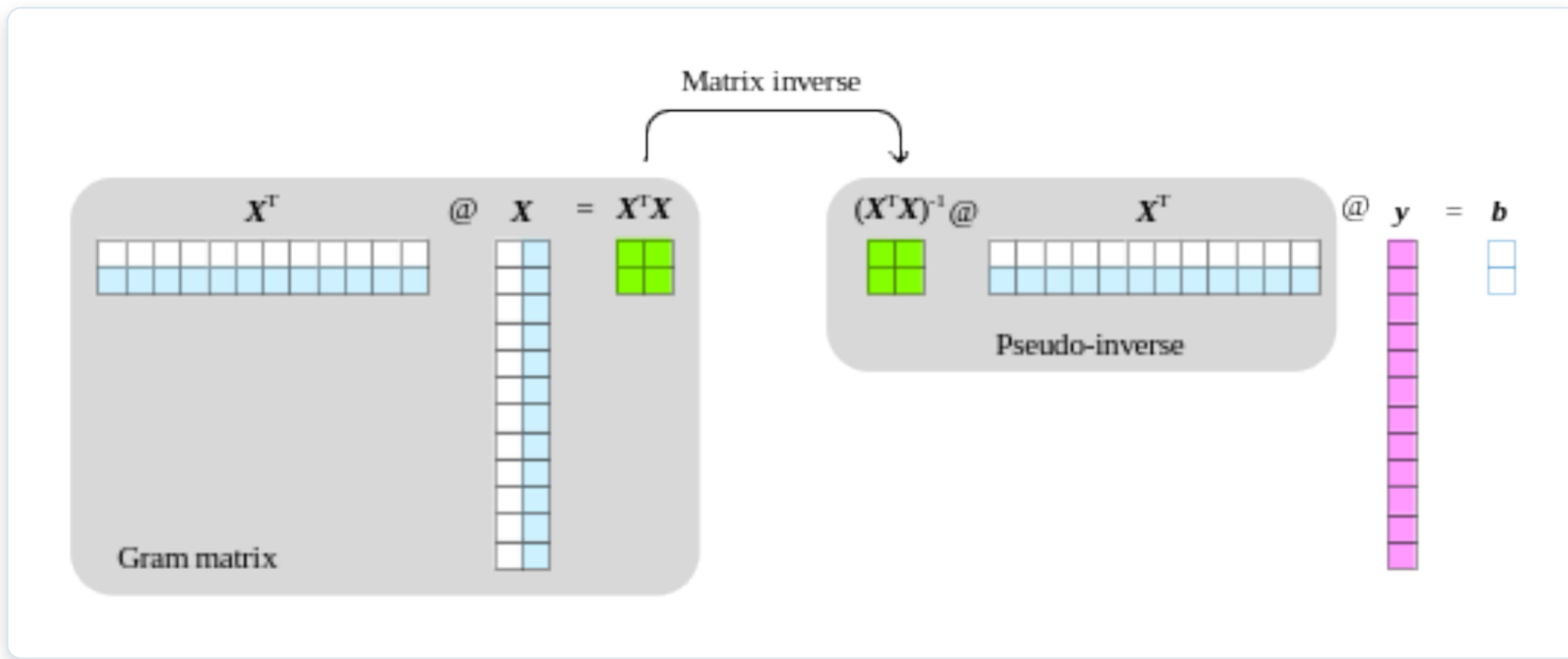
把 $\varepsilon = y - Xb$ 代入正交关系，解出最优参数 b

$$X^T X b = X^T y \Rightarrow b = (X^T X)^{-1} X^T y$$

- 1、 x_1 线性无关 $\Rightarrow$ X 列满秩 $\Rightarrow$ 格拉姆矩阵 $X^T X$ 满秩、可逆，从而解出 b 的解析式
- b 是 y 在 $\text{span}(1, x_1)$ 上投影的坐标，也就是标量投影！
- $(X^T X)^{-1} X^T$ 叫伪逆(pseudo-inverse)，也叫广义逆(generalized inverse)

图13 • 计算参数向量b的流程，一元线性回归

- 先算格拉姆矩阵 $X^T X$ 并求逆，再与 X^T 相乘得到伪逆，最后乘以 y 得到 b



算出图2中红色直线的具体参数

$$\mathbf{b} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y} = [8,41; 41,269]^{-1} [42; 261] = [1.267; 0.777]$$

- 即 $b_0=1.267$, $b_1=0.777$ (本节计算结果仅保留三位小数)
- 拟合直线的解析式为: $\hat{y}=b_0+b_1x_1=1.267+0.777x_1$

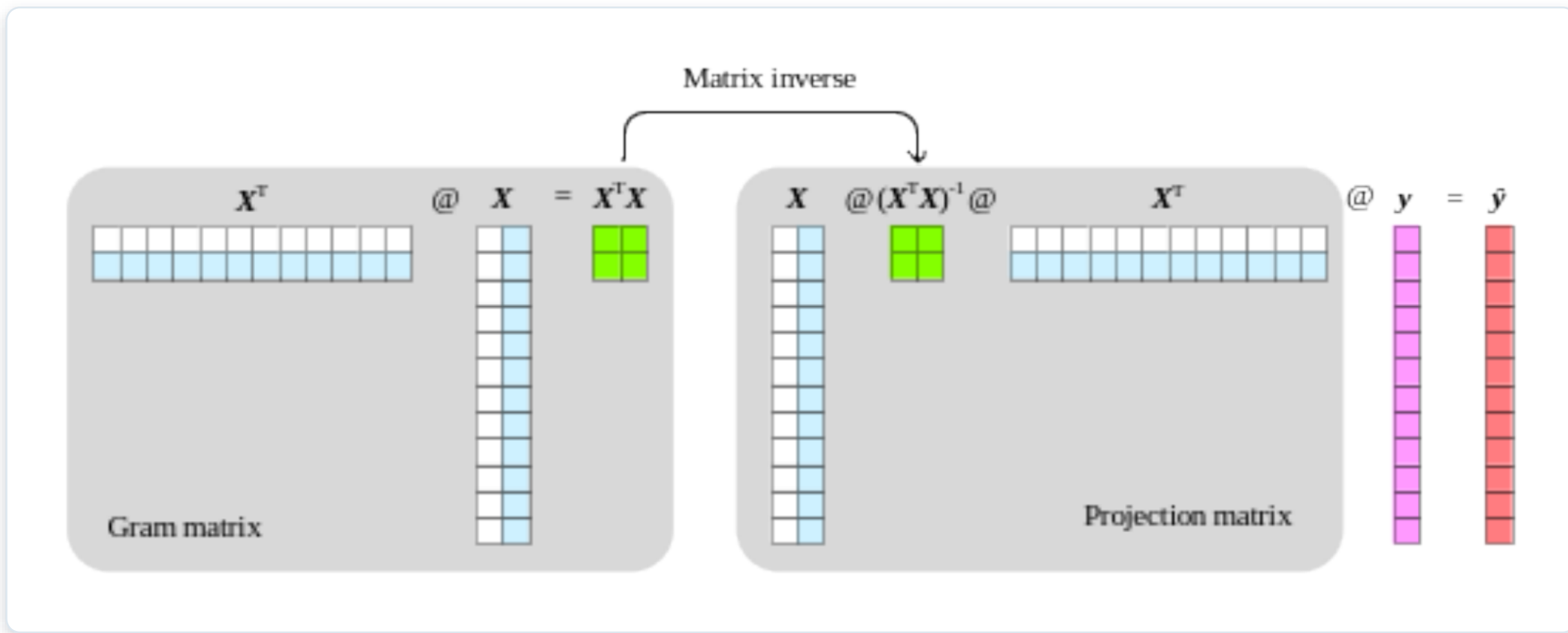
把b的解析式代入，得到投影矩阵H

$$\hat{\mathbf{y}} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} = \mathbf{H}\mathbf{y}$$

- $\mathbf{H} = \mathbf{X}(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T$ 叫做投影矩阵(projection matrix)，直接把y投影为 $\hat{\mathbf{y}}$
- 带入图1数值， $\hat{\mathbf{y}} = [2.044, 2.821, 3.598, 5.152, 5.929, 6.707, 7.484, 8.261]^T$

图14 • 计算 $\hat{y}$ 的矩阵乘法，一元线性回归

- 格拉姆矩阵求逆后，与X左右相乘得到投影矩阵，再与y相乘得到 $\hat{y}$



怎么量化残差 “尽量小” ？

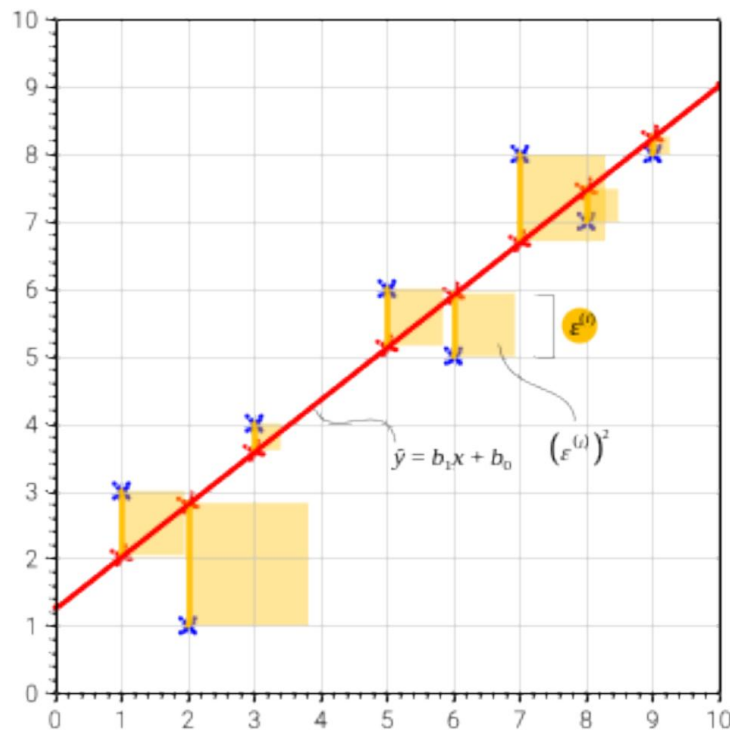
向量范数用来计算向量的 “大小” ； 最常见的是L2范数：
 $\|\epsilon\| = \|y - Xb\|$

为了方便运算，常把L2范数平方： $\|\epsilon\|^2 = \epsilon^T \epsilon = (y - Xb)^T (y - Xb) = \sum (\epsilon^{(i)})^2$

这就是最小二乘拟合(ordinary least squares, OLS)：最小化预测值与真实值误差的平方和

图15 • 残差平方之和对应图中正方形面积之和

- 每个正方形边长为残差，边长平方即 $(\varepsilon^{(i)})^2$ ；带入数值得 $||\varepsilon||^2=7.949$



怎么求解 $||\epsilon||^2$ 的最小值？

求解 $||\epsilon||^2$ 的最小值需要用到优化工具，属于《高等数学不难》要讨论的话题

获得一条红色直线并不是线性回归分析的终点，只是开始

评估模型是否显著、变量是否显著、残差是否满足正态分布和同方差性等假设，属于《概率统计不难》要讨论的话题

用Python完成一元线性回归求解

手写矩阵公式 $b = \text{inv}(X.T @ X) @ X.T @ y$ 求解最优参数

- (a) 创建列向量x、y：x是自变量，y是因变量；reshape(-1,1)转成二维列向量
- (b) 构造设计矩阵X：np.ones_like(x)造全1列，np.column_stack拼接得到X
- (c) 计算参数向量 $b = \text{inv}(X^T X) @ X^T @ y$ ——最小二乘法的标准公式
- (d) 在x_array（0到10的密集取值）上计算预测值，用于画出平滑拟合直线
- (e) 在训练数据x上做预测，得到y_pred，与真实值y对比拟合效果
- (f) 计算误差向量 $\text{error} = y - y_{\text{pred}}$ ，即每个数据点的残差
- (g) 计算 $\text{error.T} @ \text{error}$ ，即误差向量L2范数的平方，也叫残差平方和

```
## 初始化
import numpy as np
import matplotlib.pyplot as plt

## 数据
a x = np.array([1,2,3,5,6,7,8,9]).reshape(-1,1) # 自变量 x
  y = np.array([3,1,4,6,5,8,7,8]).reshape(-1,1) # 因变量 y

# 构造设计矩阵，增加全1列向量
b X = np.column_stack((np.ones_like(x), x))

## 计算参数
c b = np.linalg.inv(X.T @ X) @ X.T @ y

# 格拉姆矩阵
X.T @ X

# 格拉姆矩阵的逆
np.linalg.inv(X.T @ X)

## 计算预测值
x_array = np.linspace(0,10)
X_array = np.column_stack((np.ones_like(x_array), x_array))

d y_array_pred = X_array @ b
e y_pred = X @ b

## 误差向量
f error = y - y_pred

# 误差向量的L2范数的平方
g error.T @ error
```

用Scikit-Learn自动完成模型训练

LA_10_01_02.ipynb使用Scikit-Learn的LinearRegression工具，自动完成训练与预测

背后的数学原理与最小二乘法完全一致，结果也和手写代码完全一致

LA_10_01_01.ipynb还有一段可视化代码，用来画出散点、拟合直线，代码很简单，可自行学习

比例函数：不含截距项

一个特殊的一元线性回归——过原点的模型

如果线性回归模型采用比例函数，即截距 b_0 为0，只需要估算 b_1 ，直线过原点

此时设计矩阵 $X=x_1$ ，左侧不需要添加全1列向量1

这样很容易计算得到参数 b_1 的解析式

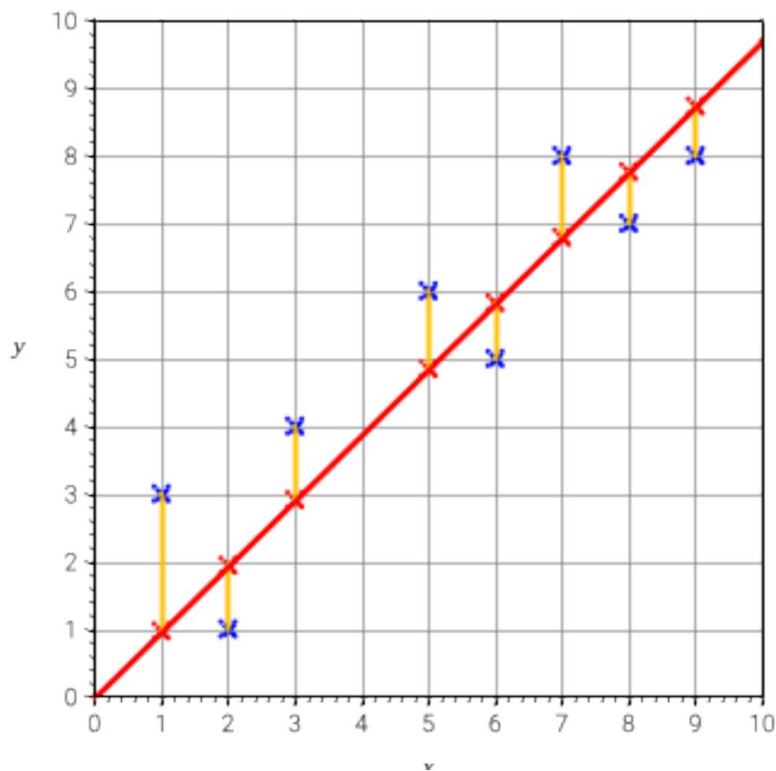
比例函数模型的参数 b_1

$$b_1 = (\mathbf{x}_1^T \mathbf{x}_1)^{-1} \mathbf{x}_1^T \mathbf{y} = 269^{-1} \times 261 = 0.970$$

- 过原点线性回归模型解析式为： $\hat{y} = b_1 x_1 = 0.970 x_1$

图16 • 比例函数模型，一元线性回归的特殊情况

- 红色直线过原点；预测值 $\hat{y}=0.970 \times [1, 2, 3, 5, 6, 7, 8, 9]^T$ ，误差平方和为10.762



过原点模型的代码实现

LA_10_01_03.ipynb求解以上截距为0（比例函数）的一元线性模型

代码结构与LA_10_01_01.ipynb几乎一致，只是设计矩阵X不再拼接全1列

从投影到回归，我们看到了什么？

一元线性回归 $\hat{y} = b_0 + b_1x_1$ ，本质上是1、 x_1 的线性组合， $\hat{y}$ 在 $\text{span}(1, x_1)$ 中

y 正交分解为 $\hat{y} + \varepsilon$ ： $\hat{y}$ 是 y 在 $\text{span}(1, x_1)$ 上的正交投影， ε 垂直该平面

写成矩阵形式 $\hat{y} = Xb$ ， $X = [1, x_1]$ 为设计矩阵，正交关系写成 $X^T \varepsilon = 0$

解出 $b = (X^T X)^{-1} X^T y$ ， $(X^T X)^{-1} X^T$ 为广义逆； $H = X(X^T X)^{-1} X^T$ 为投影矩阵

最小二乘原理：最小化残差平方和 $\|\varepsilon\|^2 = \varepsilon^T \varepsilon$ ，即OLS

过原点的比例函数模型：设计矩阵不含全1列， $b_1 = (x_1^T x_1)^{-1} x_1^T y$

本节关键术语

线性回归 *Linear Regression* —— 用线性模型描述因变量随自变量变化的规律

截距、斜率 *Intercept b_0 / Slope b_1* —— 决定直线上下位置与倾斜程度

设计矩阵 *Design Matrix X* —— 第一列常为全1，其余列为自变量取值

广义逆 *Generalized Inverse / Pseudo-Inverse* —— $(X^T X)^{-1} X^T$

最小二乘法 *Ordinary Least Squares, OLS* —— 最小化残差平方和 $\|\varepsilon\|^2$ 求解参数

自变量 / 因变量 *Independent / Dependent Variable* —— x_1 为自变量, y 为因变量

残差 *Residual ε* —— 真实值与预测值之差, $\varepsilon = y - \hat{y}$

格拉姆矩阵 *Gram Matrix* —— $X^T X$, 回归中用来求解最优参数

投影矩阵 *Projection Matrix H* —— $H = X(X^T X)^{-1} X^T$, 满足 $\hat{y} = Hy$

L2范数 *L2 Norm* —— 向量长度的常见度量, $\|\varepsilon\|^2 = \varepsilon^T \varepsilon$ 为其平方

延伸阅读

回顾 · 9.4-9.5 格拉姆-施密特正交化、QR分解

上一章 正交投影、格拉姆矩阵的代数工具

延伸阅读 · 《高等数学不难》

如何用优化工具求解 $\|\epsilon\|^2$ 的最小值

延伸阅读 · 《概率统计不难》

显著性检验、置信区间、拟合优度、多重共线性

开源仓库

全书代码与图像素材: <https://github.com/Visualize-ML>

习题Q1使用的超定方程组

设计矩阵	参数向量	观测向量
$\begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix}$	$\begin{bmatrix} b_0 & b_1 \end{bmatrix}^T$	$\begin{bmatrix} 2 & 1 & 4 & 3 \end{bmatrix}^T$

方程组 $\begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix} \begin{bmatrix} b_0 & b_1 \end{bmatrix}^T = \begin{bmatrix} 2 & 1 & 4 & 3 \end{bmatrix}^T$ 为超定方程组（方程数多于未知数），请求最小二乘解。

习题 Q1 – Q9

Q1 请手解，并编写代码求解上表超定方程组，得到最小二乘解 b_0 、 b_1 。

Q3 用matplotlib画出Q2生成数据的散点图，叠加拟合直线，观察拟合效果。

Q5 写Python函数fit_line(x, y)，返回 b_0 、 b_1 ，并在3组不同数据上测试。

Q7 计算图1数据的投影矩阵 $H = X(X^T X)^{-1} X^T$ ，验证 $H@y$ 是否等于图1的拟合值 $\hat{y}$ 。

Q9 验证：残差向量 $\varepsilon = y - Xb$ 满足 $X^T \varepsilon = 0$ ，用图1数据计算并确认数值误差在 $1e-10$ 以内。

Q2 补全代码：用给定的np.random.rand(50)*10自变量、true_slope、true_intercept生成样本数据，构造一元回归模型。

Q4 证明：若 1 、 x_1 线性相关（即 x_1 为常数向量），格拉姆矩阵 $X^T X$ 不可逆，回归系数无法唯一确定。

Q6 验证：用手写公式 $b = (X^T X)^{-1} X^T y$ 和Scikit-Learn的LinearRegression，在同一组数据上结果是否一致。

Q8 验证：投影矩阵 H 满足 $H@H = H$ （幂等性），并用代码在图1数据上验证。

习题 Q10 – Q17

Q10 探索：如果只用一个自变量的两个观测点做回归 ($n=2$)，拟合直线和残差会怎样？

Q12 对比：截距 b_0 为0（比例函数）与截距不为0的两种模型，在同一组数据上拟合效果有何不同？

Q14 探索：给数据加入一个明显的离群点，观察最小二乘拟合直线如何被拉偏。

Q16 推导：把 $\epsilon = y - Xb$ 代入 $\|\epsilon\|^2 = \epsilon^T \epsilon$ ，展开为 $y^T y - 2b^T X^T y + b^T X^T X b$ ，并对 b 求导验证正规方程。

Q11 写Python函数`residual_sum_of_squares(y, y_pred)`，计算并返回 $\|\epsilon\|^2$ 。

Q13 证明：预测值向量 $\hat{y}$ 与残差向量 ϵ 正交，即 $\hat{y}^T \epsilon = 0$ 。

Q15 写Python代码，用不同颜色标出图6中每个散点残差的正负号，观察正负残差的分布。

Q17 写Python函数`r_squared(y, y_pred)`，实现拟合优度 $R^2 = 1 - SS_{\text{res}}/SS_{\text{tot}}$ （提示：可自学后完成）。

习题 Q18 – Q25

Q18 探索：用`numpy.polyfit(x, y, 1)`完成一元线性回归，并与本节手写公式结果比较。

Q20 探索：设计矩阵 X 列数增加会怎样影响格拉姆矩阵 $X^T X$ 的形状与求逆难度？

Q22 探索：本节假设 1 、 x_1 线性无关；如果自变量本身就是常数列，会发生什么？请用代码验证。

Q24 预习思考：如果有两个自变量 x_1 、 x_2 ，设计矩阵 X 会变成什么形状？正规方程还成立吗？

Q19 写Python代码，可视化投影矩阵 H 的热图（ 8×8 矩阵），观察其结构特点。

Q21 写Python函数`standardize(x)`，对自变量做标准化后再拟合，比较 b_0 、 b_1 与原始拟合的区别。

Q23 用3组不同斜率、截距生成的数据分别拟合，比较格拉姆矩阵 $X^T X$ 的数值是否随数据变化。

Q25 预习思考：多项式回归 $\hat{y} = b_0 + b_1 x_1 + b_2 x_1^2$ 可以看作哪种设计矩阵下的线性回归？请说明理由。

10个可以和AI互动的Prompt

1 请解释为什么一元线性回归的预测值 $\hat{y}$ 可以理解为真实值 y 在 $\text{span}(1, x_1)$ 上的正交投影。

2 请解释设计矩阵 X 为什么要在第一列添加全1列向量，这一列代表什么含义？

3 请解释正规方程 $X^T X b = X^T y$ 是如何从正交关系 $X^T \varepsilon = 0$ 推导出来的。

4 请解释为什么 $(X^T X)^{-1} X^T$ 被称为伪逆、广义逆，它和方阵的普通逆矩阵有什么关系？

5 请用Python演示手写最小二乘公式和Scikit-Learn的LinearRegression结果是否一致。

10个可以和AI互动的Prompt

6 请解释投影矩阵 $H = X(X^T X)^{-1} X^T$ 为什么满足 $H @ H = H$ ，这在几何上意味着什么？

7 请解释为什么最小二乘法要最小化残差平方和，而不是残差绝对值之和？

8 请解释过原点的比例函数模型与含截距项的一般模型，设计矩阵有什么区别？

9 请解释残差 ϵ 与预测值 $\hat{y}$ 为什么互相正交，这对模型评估有什么意义？

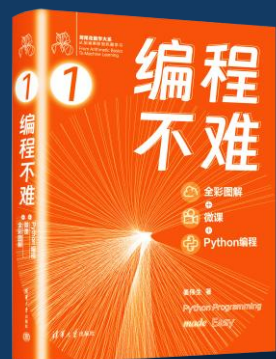
10 请解释线性回归为什么不能说明因果关系，只能说明相关关系？

数学

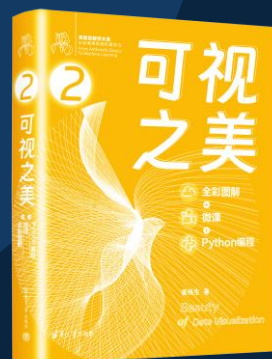
数学基础

线性代数

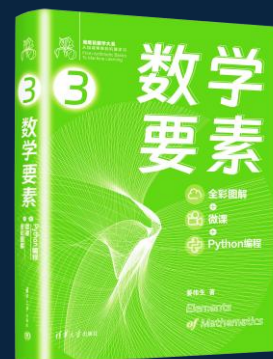
概率统计



Python编程



数据可视化



工具

数据分析
图和网络

回归、降维
分类、聚类

实践

